Spec-Driven Development

Lets Talk

Grégoire Mercier · bluehands

bluehands
()=>SoftwareEntwicklung()

Worum's heute geht

1. Warum Spec Driven
2. Der Workflow
3. Agents dazu bringen, mein Ding zu machen

Von „Vibe Coding“ zu was Robusterem

- Schwierig bei komplexen Aufgaben
- Schwierig bei Absprachen zwischen Agenten
- Anforderungen **explizit** und **agentenlesbar**

Mein Arbeitskontext

- Anforderungen und Features in **Azure DevOps**
- **.NET**, bestehender Code, bestehende Tests
- Build, Analyzer und Tests als Gates

Spec Kit und OpenSpec

Spec Kit

- Spec, Plan, Tasks
- Phasen als Orientierung
- Constitution für Projektregeln

OpenSpec

- Specs und Änderungen getrennt
- Iteratives Arbeiten
- Given/When/Then für Szenarien

Bei mir: eigene Skills mit Übergabe an Azure DevOps

Vier Skills

Schritt	Ergebnis
`/spec-design`	`requirements.md`
`/spec-plan`	`plan.md`
`/spec-prepare`	`pbis.md`, nach Freigabe ADO-PBIs
`/spec-implement`	Lokaler Branch, Code und Tests

Explizite menschliche Übergabe zwischen den einzelnen Schritten.

Iterative Verbesserung

- Hätte eine Regel einen **konkreten Vorfall** verhindert?
- Kostet sie mehr Zeit, als sie spart?
- Versteh ich in drei Monaten noch, warum sie da ist?

Spezifikationsformate

`requirements.md`

Anforderungen

- Context, Goals, Non-Goals
- Functional Requirements
- Non-Functional Requirements

Einordnung

- Architekturentscheidungen
- Risiken und offene Fragen
- **Code Surface**

Dazu: Historie, Glossar, Referenzen

SHALL, SHOULD, MAY

```
- **FR-1** `[SHALL]`  Leere Exporte enthalten nur die Kopfzeile.
- **FR-2** `[SHOULD]` Der Export zeigt einen Fortschrittsbalken.
- **FR-3** `[MAY]`    Der Export kann pausiert werden.
```

- **SHALL**: ohne das ist das Feature nicht fertig
- **SHOULD**: kommt rein, außer Zeit oder Risiko sprechen dagegen
- **MAY**: optional

Given/When/Then

Given ein leerer Datensatz

When ich exportiere

Then eine Datei nur mit Kopfzeile, kein Fehler

Andere Kriterien als Bullets:

- Audit-Log-Eintrag für jeden Export

Code Surface

Bestehender Code

- Betroffene Dateien
- Wiederverwendbarer Code
- Tests und Konventionen
- Einstiegspunkte

Neues Feature ohne Code

- Geplante Zielpfade
- `greenfield: true`
- Noch keine Dateien anlegen

Pro Eintrag: `path`, `reason`, `confidence`, `greenfield`

Ein Plan-Item (Ausschnitt)

```yaml
id: S-1
type: story
title: Leere Datensätze als CSV exportieren
refs: [FR-1]
ac_behavioral:
  - "Given ein leerer Datensatz, When ich exportiere,
     Then entsteht eine Datei nur mit Kopfzeile."
```

Feste Felder für die Übergabe.

Das Ticket als Arbeitsauftrag

Azure DevOps

- PBI mit Akzeptanzkriterien
- Zuordnung zum Feature
- Feature-AC über alle PBIs abdecken

Im Repository

- `pbis.md` ist Pflicht und Arbeitsauftrag
- Erst Entwurf, dann freigegebener Inhalt
- ADO-Abgleich bei vorhandener ID

Agents dazu bringen, mein Ding zu machen

Vom PBI zum Code

Design freigeben

- Typen, Schnittstellen, Abhängigkeiten
- Testplan für die Akzeptanzkriterien
- Meine Freigabe vor der Umsetzung

Test-first umsetzen

- Test schreiben, Fehlschlag prüfen
- Implementieren, dann refaktorieren
- Build, Tests und lokale Commits

Review mit frischem Kontext

Self-Critique

- 2–3 kritische Agenten
- Begründung darf bekannt sein
- Ergebnisse vor der Vorstellung einarbeiten

Unabhängige Prüfung

- Diff und PBI-Inhalt
- Keine Design-Notiz oder Vorbefunde
- Eigene Bewertung

Getrennt: Code-Review, Security-Review, AC-Mapping

Akzeptanzkriterien prüfen

Prüfung	Frage
Lokale Zuordnung	Welcher Code und Test erfüllt welches Kriterium?
ADO-Abgleich, falls möglich	Stimmen die Kriterien mit dem Work Item überein?
Frisches Review	Trägt die Zuordnung auch bei erneuter Prüfung?

Kein PROVEN nach dem AC-Mapping: Übergabe stoppen

Grenzen des frischen Kontexts

Bewusst zurückgehalten

- Design-Begründung
- Vorherige Review-Befunde
- AC-Matrix des Implementierers

Gemeinsame Fehlerquellen

- Dieselbe unvollständige Spec
- Fehlendes Fachwissen im PBI
- Ähnliche Annahmen der Modelle

Der Reviewer kennt die Anforderung, aber nicht die Begründung der Umsetzung.

Aufwand der Prüfungen

- Zusätzliche Reviews und Rückfragen
- Wartezeit auch bei kleinen Änderungen
- Mehr Regeln, die ich selbst pflegen muss

Weniger hartes Hinterfragen: `ease up`

Übergabe nach der Implementierung

Ergebnis des Skills

- Lokaler Branch und Commits
- Tests und Review-Befunde
- AC-Matrix und Follow-ups

Manuell

- Remote-Stand holen und integrieren
- Branch pushen, PR öffnen
- ADO-Status und Links pflegen

Noch offene Fragen

- Wer hält Spec und Code langfristig synchron?
- Wie viel Review braucht eine kleine Änderung?
- Wann wird aus meinen Skills doch ein Framework?

Und ihr so?

Welche Erfahrungen habt ihr mit SDD?

Grégoire Mercier · gregoire.mercier@bluehands.de